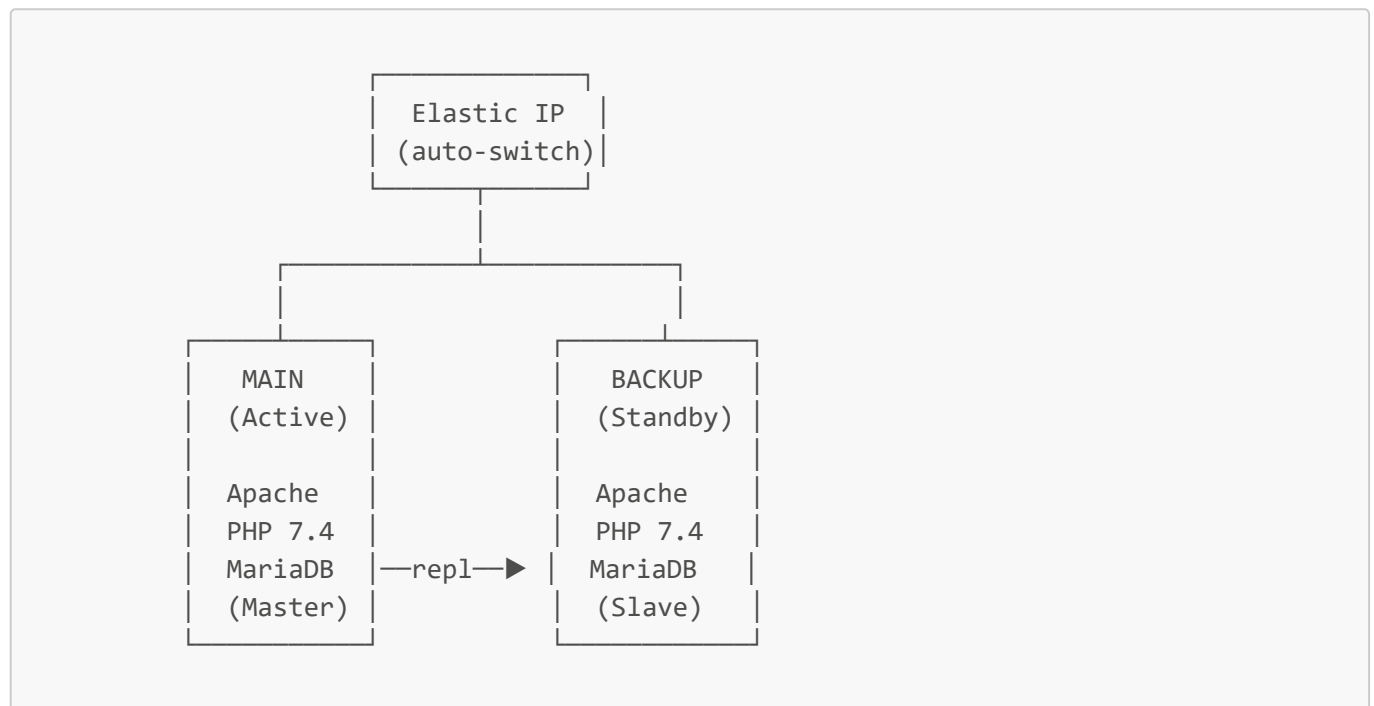


AWS Active-Passive Failover Setup Guide

800 CarGuru (CG2) - High Availability Configuration

Date: 2026-04-01 **Instance:** SMF_NEW_CG (t3.2xlarge) **Region:** us-east-1f **Domain:** 800carguru.me

Architecture Overview



How It Works

1. **Main server** handles all traffic via Elastic IP
 2. **Backup server** stays idle but receives real-time database replication
 3. If main goes down, backup auto-detects and takes over the Elastic IP
 4. Users experience minimal downtime (~1-2 minutes)
 5. When main recovers, failback is performed manually
-

Part 1: Create Backup Instance

1.1 Create AMI from Main Instance

1. Go to **EC2 Console** → Select main instance
2. **Actions** → **Image and templates** → **Create image**
3. Image name: **CG-backup-base**
4. Description: **Base image for failover backup**
5. Enable **No reboot** to avoid downtime
6. Click **Create image**
7. Wait for AMI status to become **available** (5-15 minutes)

1.2 Launch Backup Instance from AMI

- 1. Go to **AMIs** → Select **CG-backup-base** → **Launch instance from AMI**
- 2. Configure:
 - **Name:** **SMF_BACKUP_CG**
 - **Instance type:** **t3.2xlarge** (or **t3.large** to save cost)
 - **Key pair:** Same as main instance
 - **VPC:** **vpc-0423103401dfdca5f**
 - **Subnet:** Same AZ (us-east-1f)
 - **Security group:** Same as main instance
 - **Storage:** Same as main (500 GiB gp3)
- 3. **Advanced Details:**
 - Termination protection: Enable
 - Stop protection: Enable
 - IMDSv2: Required
- 4. Click **Launch instance**

1.3 Assign Elastic IP to Main Instance

- 1. Go to **EC2** → **Elastic IPs** → **Allocate Elastic IP address**
- 2. Click **Allocate**
- 3. Select the new Elastic IP → **Actions** → **Associate Elastic IP address**
- 4. Select **main instance** → **Associate**
- 5. Update DNS (GoDaddy) A record to point to the Elastic IP

Important: Note down the Elastic IP **Allocation ID** — needed for failover script.

Part 2: Database Replication Setup

2.1 Security Group Configuration

Add inbound rule on **both** instances' security groups:

Type	Port	Source
MySQL/Aurora	3306	Main private IP (e.g., 172.31.38.189/32)
MySQL/Aurora	3306	Backup private IP (e.g., 172.31.x.x/32)

2.2 Configure Main Server (Master)

SSH into the **main** server:

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Add under **[mysqld]** section:

```
server-id = 1
log_bin = /var/log/mysql/mysql-bin.log
binlog_do_db = carguru2
bind-address = 0.0.0.0
```

Restart MariaDB:

```
sudo systemctl restart mariadb
```

Create replication user:

```
sudo mysql -e "
CREATE USER 'replicator'@'%' IDENTIFIED BY 'YourStrongPassword123';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'%';
FLUSH PRIVILEGES;
"
```

Get master status (note the File and Position values):

```
sudo mysql -e "SHOW MASTER STATUS;"
```

Example output:

```
+-----+-----+-----+
| File           | Position | Binlog_Do_DB |
+-----+-----+-----+
| mysql-bin.000001 |      328 | carguru2      |
+-----+-----+-----+
```

Save these values — needed for backup server setup.

2.3 Configure Backup Server (Slave)

SSH into the **backup** server:

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Add under `[mysqld]` section:

```
server-id = 2
relay-log = /var/log/mysql/mysql-relay-bin.log
read_only = 1
```

Restart MariaDB:

```
sudo systemctl restart mariadb
```

Configure replication (replace values from master status):

```
sudo mysql -e "
CHANGE MASTER TO
  MASTER_HOST='<MAIN_PRIVATE_IP>',
  MASTER_USER='replicator',
  MASTER_PASSWORD='YourStrongPassword123',
  MASTER_LOG_FILE='mysql-bin.000001',
  MASTER_LOG_POS=328;
START SLAVE;
"
```

2.4 Verify Replication

```
sudo mysql -e "SHOW SLAVE STATUS\G"
```

Confirm these two values are both **Yes**:

```
Slave_IO_Running: Yes
Slave_SQL_Running: Yes
```

2.5 Test Replication

On **main** server:

```
sudo mysql carguru2 -e "CREATE TABLE repl_test (id INT); INSERT INTO repl_test
VALUES (1);"
```

On **backup** server:

```
sudo mysql carguru2 -e "SELECT * FROM repl_test;"
```

If it returns **1**, replication is working. Clean up:

```
# On main server
sudo mysql carguru2 -e "DROP TABLE repl_test;"
```

Part 3: Auto-Failover Script

3.1 Install AWS CLI on Backup Server

```
sudo apt install -y awscli
aws configure
```

Enter your AWS Access Key, Secret Key, and region (**us-east-1**).

3.2 Create Failover Script

On the **backup** server:

```
sudo nano /home/ubuntu/failover.sh
```

Paste the following (replace placeholder values):

```
#!/bin/bash

# =====
# AUTO-FAILOVER SCRIPT
# Runs every minute via cron on BACKUP server
# =====

MAIN_IP="<MAIN_PRIVATE_IP>"           # e.g., 172.31.38.189
ELASTIC_IP="<YOUR_ELASTIC_IP>"         # e.g., 34.199.53.251
INSTANCE_ID="<BACKUP_INSTANCE_ID>"     # e.g., i-0abc123def456
ALLOCATION_ID="<ELASTIC_IP_ALLOC_ID>"  # e.g., eipalloc-0abc123
FAILOVER_FLAG="/tmp/failover_active"
LOG_FILE="/var/log/failover.log"

# If failover is already active, skip
if [ -f "$FAILOVER_FLAG" ]; then
    exit 0
fi

# Check 1: Ping main server
if ping -c 3 -W 2 $MAIN_IP > /dev/null 2>&1; then
    exit 0
```

```

fi

# Check 2: Wait 30 seconds and try again (avoid false positives)
sleep 30

if ping -c 3 -W 2 $MAIN_IP > /dev/null 2>&1; then
    exit 0
fi

# Check 3: HTTP check
if curl -s --max-time 5 http://$MAIN_IP > /dev/null 2>&1; then
    exit 0
fi

# === MAIN SERVER IS DOWN - ACTIVATE FAILOVER ===

echo "$(date) - FAILOVER TRIGGERED - Main server is DOWN" >> $LOG_FILE

# Move Elastic IP to backup instance
aws ec2 associate-address \
    --instance-id $INSTANCE_ID \
    --allocation-id $ALLOCATION_ID \
    --allow-reassociation \
    --region us-east-1

if [ $? -eq 0 ]; then
    echo "$(date) - Elastic IP moved to backup instance" >> $LOG_FILE

    # Stop slave replication and make database writable
    sudo mysql -e "STOP SLAVE; SET GLOBAL read_only = 0;"

    echo "$(date) - Database is now writable" >> $LOG_FILE

    # Create flag to prevent re-triggering
    touch $FAILOVER_FLAG

    echo "$(date) - FAILOVER COMPLETE - Backup is now ACTIVE" >> $LOG_FILE
else
    echo "$(date) - ERROR: Failed to move Elastic IP" >> $LOG_FILE
fi

```

Make executable:

```
sudo chmod +x /home/ubuntu/failover.sh
```

3.3 Schedule Failover Check (Every Minute)

```
sudo crontab -e
```

Add:

```
* * * * * /home/ubuntu/failover.sh
```

Part 4: Failback Procedure (Manual)

When the main server is back online, follow these steps:

4.1 Sync New Data from Backup to Main

On **backup** server, get current binlog position:

```
sudo mysql -e "SHOW MASTER STATUS;"
```

On **main** server:

```
sudo mysql -e "  
CHANGE MASTER TO  
  MASTER_HOST='<BACKUP_PRIVATE_IP>',  
  MASTER_USER='replicator',  
  MASTER_PASSWORD='YourStrongPassword123',  
  MASTER_LOG_FILE='<backup_file>',  
  MASTER_LOG_POS=<backup_position>;  
START SLAVE;  
"
```

Wait for main to catch up:

```
sudo mysql -e "SHOW SLAVE STATUS\G" | grep "Seconds_Behind_Master"
```

When **Seconds_Behind_Master: 0**, proceed.

4.2 Switch Back to Main

Move Elastic IP back to main:

```
aws ec2 associate-address \  
  --instance-id <MAIN_INSTANCE_ID> \  
  --allocation-id <ELASTIC_IP_ALLOC_ID> \  
  --allow-reassociation \  
  --region us-east-1
```

4.3 Restore Replication Direction

On **main** server:

```
sudo mysql -e "STOP SLAVE; RESET SLAVE ALL;"
```

On **backup** server:

```
sudo mysql -e "  
SET GLOBAL read_only = 1;  
CHANGE MASTER TO  
  MASTER_HOST='<MAIN_PRIVATE_IP>',  
  MASTER_USER='replicator',  
  MASTER_PASSWORD='YourStrongPassword123',  
  MASTER_LOG_FILE='<main_file>',  
  MASTER_LOG_POS=<main_position>;  
START SLAVE;  
"
```

4.4 Remove Failover Flag

On **backup** server:

```
rm /tmp/failover_active
```

Part 5: Database Backup (Daily)

5.1 Backup Script on Main Server

```
sudo nano /home/ubuntu/db_backup.sh
```

```
#!/bin/bash  
BACKUP_DIR="/home/ubuntu/db_backups"  
DATE=$(date +%Y-%m-%d_%H-%M-%S)  
mkdir -p $BACKUP_DIR  
  
# Backup database  
mysqldump -u root carguru2 | gzip > "$BACKUP_DIR/carguru2_$DATE.sql.gz"  
  
# Keep only last 7 days  
find $BACKUP_DIR -name "*.sql.gz" -mtime +7 -delete
```



```
sudo chmod +x /home/ubuntu/db_backup.sh
```

5.2 Schedule Daily Backup (2 AM)

```
sudo crontab -e
```

Add:

```
0 2 * * * /home/ubuntu/db_backup.sh
```

5.3 Auto-Download to Local PC (Windows)

Create `D:\DB_BKP\auto_backup.bat`:

```
@echo off
set KEY="C:\path\to\your-key.pem"
set SERVER=ubuntu@<ELASTIC_IP>

scp -i %KEY% %SERVER%:/home/ubuntu/db_backups/*.sql.gz D:\DB_BKP\
```

Schedule in **Windows Task Scheduler** to run daily at 3 AM.

Monitoring & Health Checks

Check Replication Status

```
# On backup server
sudo mysql -e "SHOW SLAVE STATUS\G" | grep -E "Slave_IO|Slave_SQL|Seconds_Behind"
```

Check Failover Log

```
cat /var/log/failover.log
```

Check if Failover is Active

```
ls /tmp/failover_active
```

Cost Estimate

Component	Monthly Cost
Main (t3.2xlarge)	~\$240
Backup (t3.2xlarge)	~\$240
Backup (t3.large)*	~\$60
Elastic IP	Free
Extra EBS (500GB gp3)	~\$40
Total (full)	~\$520
Total (cost-saving)	~\$340

*Use t3.large for backup to save cost — sufficient for temporary traffic handling.

Important Notes

1. **Always assign Elastic IP before updating DNS** — prevents IP changes on restart
 2. **Test failover monthly** — stop main instance intentionally and verify backup takes over
 3. **Monitor replication lag** — if `Seconds_Behind_Master` grows, investigate
 4. **Keep security group rules tight** — only allow replication between the two instances
 5. **Update the replication password** — change `YourStrongPassword123` to a strong password
 6. **Replace all `<PLACEHOLDER>` values** with actual IPs, instance IDs, and allocation IDs
-

Quick Reference

Item	Value
Main Instance ID	<code><fill in></code>
Backup Instance ID	<code><fill in></code>
Elastic IP	<code><fill in></code>
Elastic IP Alloc ID	<code><fill in></code>
Main Private IP	<code><fill in></code>
Backup Private IP	<code><fill in></code>
Domain	800carguru.me
Database	carguru2
Replication User	replicator
VPC	vpc-0423103401dfdca5f

Item	Value
Region/AZ	us-east-1f